// DEEP DIVE

Linux for SOC Analysts

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

The Linux fundamentals a SOC analyst actually needs — filesystem layout, permissions, where every log actually lives, and the command-line triage flow for a host you suspect is compromised.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	Why Linux matters for a SOC role, and the filesystem hierarchy
3 — 4	Permissions/ownership, and users, groups & sudo
5 — 7	Where logs live, and command-line triage of processes, network & files
8 — 9	A full worked triage of a suspicious host, and reference

How to use this guide: read start to finish for the full picture, or jump to section 8 for the worked triage walkthrough. Every command shown is meant to be run and read, not just memorized.

// INDEX

Table of Contents

1. Why Linux Matters for a SOC Analyst 3
2. The Filesystem Hierarchy Standard 4
3. Permissions & Ownership 5
4. Users, Groups & sudo 6
5. Where the Logs Live 7
6. Command-Line Triage: Processes & Network 8
7. Command-Line Triage: Files & Persistence 9
8. Worked Example: Triaging a Suspicious Host 10
9. Quick Reference & Glossary 11

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

Why Linux Matters for a SOC Analyst

1.1 IT'S EVERYWHERE YOU'LL ACTUALLY BE WORKING

Most servers, cloud infrastructure, and a large share of the security tooling a SOC analyst uses daily — SIEM collectors, IDS sensors, log forwarders, the underlying host for a Wazuh or Splunk deployment (see the SIEM & Log Analysis guide) — run Linux. Being fluent at a Linux command line isn't an optional extra skill; it's the baseline environment a huge fraction of real investigative and triage work actually happens in.

1.2 WHAT "FLUENT" ACTUALLY MEANS HERE

- **Knowing where things live** — logs, configuration, and evidence of compromise all have conventional locations (Sections 2 and 5); knowing them by memory is far faster than searching.
- **Reading permissions correctly** — understanding exactly who can read, write, or execute a file is central to both investigating misuse and recognizing suspicious changes (Section 3).
- **Comfort with the command line under pressure** — triage often happens live, on a system you suspect is actively compromised, without the luxury of a GUI or extensive documentation lookup time (Sections 6–7).

> **This guide assumes basic command-line familiarity.** It's not a "how to use a terminal" primer — it's specifically the Linux knowledge that matters for security triage and investigation, building on the assumption you can already navigate a shell.

// SECTION 2

The Filesystem Hierarchy Standard

2.1 ONE STANDARD LAYOUT, EVERYWHERE

Nearly every Linux distribution follows the **Filesystem Hierarchy Standard (FHS)** — a consistent, predictable directory layout. Knowing this layout by heart means you can navigate an unfamiliar system's filesystem with the same confidence as one you use daily, which matters enormously when triaging a host you've never touched before.

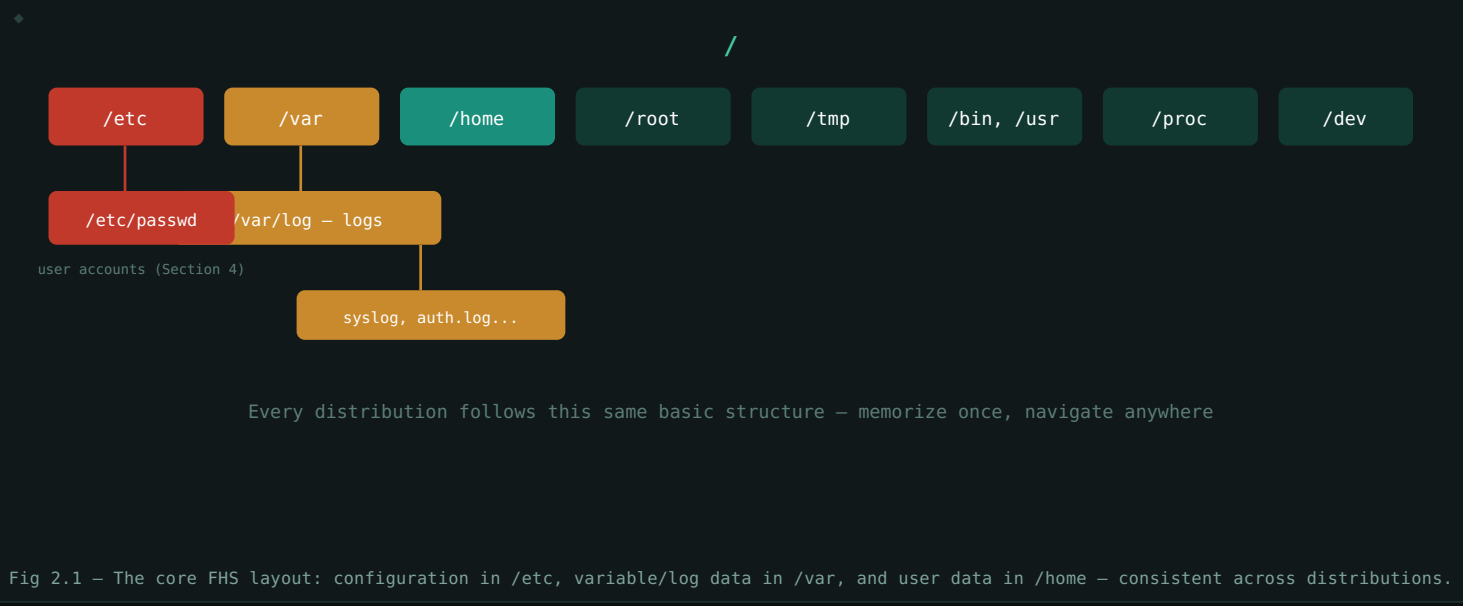

Fig 2.1 — The core FHS layout: configuration in /etc, variable/log data in /var, and user data in /home — consistent across distributions.

2.2 DIRECTORY REFERENCE

DIRECTORY	CONTAINS
`/etc`	System-wide configuration files, including `/etc/passwd` and `/etc/shadow` (Section 4)
`/var`	Variable data — most importantly `/var/log` (Section 5)
`/home`	Per-user home directories — `/home/username`
`/root`	The root user's home directory (not inside `/home`)
`/tmp`	Temporary files, world-writable — a common landing spot for malware and attacker tooling
`/bin`, `/usr/bin`	Executable programs
`/proc`	A virtual filesystem exposing live kernel and process information (Section 6)
`/dev`	Device files representing hardware and virtual devices

/tmp deserves special attention during triage. Because it's world-writable by default and often excluded from strict application allow-listing, it's a very common location for dropped malware, staged exfiltration data, or attacker scripts — checking it should be a near-automatic step in any Linux host investigation.

// SECTION 3

Permissions & Ownership

3.1 READING THE PERMISSION STRING

Every file has three permission sets — for its **owner**, its **group**, and **everyone else** — each made of **read (r)**, **write (w)**, and **execute (x)** flags, shown together as a 10-character string like `-rwxr-xr--`.

Fig 3.1 — Reading a permission string: type, then three rwx triplets for owner, group, and everyone else, each also expressible as a single digit.

3.2 THE NUMERIC (OCTAL) SHORTHAND

Each triplet can be written as one digit: read=4, write=2, execute=1, summed together — so `rwx`=7, `r-x`=5, `r--`=4. The example above, `rwxr-xr--`, is `754` — the notation you'll see constantly in both documentation and command usage like `chmod 754 file`.

3.3 SUID, SGID & THE STICKY BIT

SPECIAL BIT	EFFECT	SECURITY RELEVANCE
SUID	Executable runs with the *file owner's* privileges, not the invoking user's	A SUID root binary is a classic privilege escalation target if misconfigured or vulnerable
SGID	Executable runs with the file's group privileges; on a directory, new files inherit its group	Less commonly abused, but still worth noting on unusual files
Sticky Bit	On a directory, only a file's owner (or root) can delete/rename it, even if others can write there	Standard on `/tmp` specifically to limit this exact abuse

Hunting for SUID binaries is a real triage step. The command `find / -perm -4000 -type f 2>/dev/null` lists every SUID binary on a system. An unexpected SUID binary — especially one outside standard system directories — is a strong indicator worth investigating immediately, since it's a well-known privilege escalation technique.

// SECTION 4

Users, Groups & sudo

4.1 WHERE ACCOUNT INFORMATION LIVES

FILE	CONTAINS
`/etc/passwd`	Every user account: username, UID, GID, home directory, default shell — world-readable
`/etc/shadow`	Hashed passwords and password aging policy — readable only by root
`/etc/group`	Group definitions and their members
`/etc/sudoers`	Rules defining who can run commands as another user (typically root) via sudo

4.2 READING /ETC/PASSWD FOR SIGNS OF TROUBLE

Each line follows the format `username:x:UID:GID:comment:home:shell`. A few things worth specifically checking during triage: any account with **UID 0** other than the legitimate `root` entry (a classic backdoor technique — creating a second root-equivalent account), any unfamiliar username, and any account with an interactive shell (`/bin/bash`) that should logically have a non-login shell like `/usr/sbin/nologin` instead.

```
$ awk -F: '$3 == 0 {print $1}' /etc/passwd
$ root
```

This command lists every account with UID 0 — on a clean system, only `root` should appear. Any second name here is an urgent finding.

4.3 SUDO: ELEVATED PRIVILEGE, LOGGED

`sudo` lets an authorized user run specific commands (or all commands) as another user, most commonly root, without ever needing the root password directly. Critically for investigation purposes, sudo usage is logged — typically to `/var/log/auth.log` (Debian/Ubuntu) or `/var/log/secure` (RHEL/CentOS), covered fully in Section 5 — making it possible to reconstruct exactly who elevated privileges, when, and to run what command.

// SECTION 5

Where the Logs Live

5.1 THE /VAR/LOG DIRECTORY

Nearly all system and security-relevant logging lands under `/var/log` (Section 2.2). Exact filenames vary somewhat between distribution families, but the categories are consistent.

LOG	LOCATION (DEBIAN/UBUNTU)	LOCATION (RHEL/CENTOS)	CONTAINS
Authentication	`/var/log/auth.log`	`/var/log/secure`	Logins, sudo usage, SSH connections
General system	`/var/log/syslog`	`/var/log/messages`	General system and kernel messages
Package management	`/var/log/dpkg.log`	`/var/log/yum.log`	Software installed, removed, or updated
Cron	`/var/log/cron.log`	within `/var/log/cron`	Scheduled task execution — a common persistence check point (Section 7)
Web server	`/var/log/apache2/` or `/var/log/nginx/`		HTTP access and error logs

5.2 SYSTEMD'S JOURNAL: A MODERN ALTERNATIVE

On systemd-based distributions, much of this same information is also captured in the **systemd journal**, queried with `journalctl` rather than reading flat log files directly. It offers powerful filtering (by time range, service, priority) directly at the command line:

```
$ journalctl -u sshd --since "1 hour ago"
```

5.3 THE FIRST RULE OF LOG TRIAGE: ASSUME THEY MIGHT BE TAMPERED WITH

An attacker with root access can edit or delete logs. Log timestamps that don't line up, unexplained gaps in otherwise continuous logging, or logs that appear suspiciously "too clean" around the suspected incident window are themselves findings, not just an absence of evidence. This is exactly why centralized, remote log forwarding to a SIEM (see the SIEM & Log Analysis guide) matters — logs shipped off-host before an attacker gains full control are far harder to tamper with retroactively.

// SECTION 6

Command-Line Triage: Processes & Network

6.1 WHAT'S ACTUALLY RUNNING

```
$ ps aux --forest
$ ps -eo pid,ppid,user,cmd,etime --sort=-etime
```

The `--forest` view shows parent-child process relationships, useful for spotting a process spawned from somewhere it shouldn't be (a shell spawned by a web server process, for instance). Sorting by elapsed time (`etime`) helps surface either very new processes (often relevant right after a suspected compromise) or unusually long-running ones.

6.2 WHAT'S LISTENING & CONNECTED

```
$ ss -tulnp
$ ss -tanp | grep ESTAB
```

`ss` (the modern replacement for `netstat`) shows listening ports and their owning process (`-tulnp`) and active established connections (`-tanp | grep ESTAB`) — an unexpected listening port, or an established connection to an unfamiliar external IP, is exactly the kind of finding this step exists to surface.

6.3 CROSS-REFERENCING PROCESS ↔ NETWORK ↔ FILE

```
$ lsof -p <PID>
```

`lsof` ("list open files") shows every file, socket, and network connection a specific process currently has open — the single most useful command for turning "this process looks suspicious" into "here's exactly what it's touching," connecting Section 6.1's process list to Section 6.2's network view and Section 7's file-level checks.

> **Living-off-the-land is common on Linux too.** Attackers frequently use entirely legitimate system binaries (`curl`, `bash`, `python`) for malicious purposes rather than dropping obviously suspicious custom malware — this is why *context* (who spawned it, what it's connected to, when it started) matters more than the process name alone when triaging.

// SECTION 7

Command-Line Triage: Files & Persistence

7.1 FINDING RECENTLY MODIFIED FILES

```
$ find / -type f -mtime -1 -not -path "/proc/*" 2>/dev/null
```

Lists files modified within the last day, excluding the noisy virtual `/proc` filesystem — a quick way to see what's changed recently on a host, especially useful when you have even an approximate timeframe for a suspected compromise.

7.2 COMMON LINUX PERSISTENCE MECHANISMS

MECHANISM	WHERE TO CHECK
Cron jobs	`crontab -l` (per-user), `/etc/cron.d/`, `/etc/crontab`
systemd services	`systemctl list-units --type=service`, `/etc/systemd/system/`
Shell startup scripts	`~/.bashrc`, `~/.bash_profile`, `/etc/profile.d/`
SSH authorized keys	`~/.ssh/authorized_keys` — an added key grants persistent remote access
SUID binaries	`find / -perm -4000` (Section 3.3) — an unusual one may itself be the persistence

7.3 CHECKING SSH AUTHORIZED_KEYS SPECIFICALLY

```
$ find / -name "authorized_keys" -exec ls -la {} \;
$ cat ~/.ssh/authorized_keys
```

An attacker-added public key here grants ongoing, password-free remote access, without needing to maintain any other foothold — checking every user's `authorized_keys` file for unrecognized entries is a standard, fast persistence check.

> **Compare against a known-good baseline whenever possible.** Every check in this section is far more powerful when you have something to compare against — a configuration management record (see the SDN & Automation guide's Infrastructure as Code coverage), a previous audit, or a golden image. Without a baseline, you're relying entirely on "does this look wrong," which is far less reliable than "this doesn't match what should be here."

// SECTION 8

Worked Example: Triaging a Suspicious Host

A SIEM alert (see the SIEM & Log Analysis guide) flags unusual outbound traffic from a Linux web server. Walking the triage flow built across this guide:

1. **Check active connections (Section 6.2):** `ss -tanp | grep ESTAB` reveals an established connection to an unfamiliar external IP on an unusual port.
2. **Identify the owning process (Section 6.2 → 6.3):** the connection's PID traces to a process named `apache2` — but running from an unexpected path, not the standard install location.
3. **Inspect that process with lsof (Section 6.3):** it has an open file handle to a script sitting in `/tmp` (Section 2.2's note about /tmp being a common landing spot proves relevant here).
4. **Check the file's permissions and timestamp (Section 3, 7.1):** the script is world-executable and was modified two hours ago — right around the alert's timeframe.
5. **Check for persistence (Section 7.2):** a cron entry in `/etc/cron.d/` references the same script, set to re-run every 15 minutes — the actual persistence mechanism.
6. **Check accounts and SSH keys (Section 4.2, 7.3):** `/etc/passwd` checks clean (no rogue UID 0 account), but an unrecognized key has been added to `root`'s `authorized_keys` — a second, independent persistence mechanism.

> **Root cause identified:** a malicious script was dropped in `/tmp`, likely via a web application vulnerability given the process ancestry, with two separate persistence mechanisms (cron and an SSH key) established afterward. Note how many earlier sections' specific checks — network, process, file, cron, SSH keys — each contributed one piece; no single command found the whole picture alone.

// SECTION 9

Quick Reference & Glossary

TERM	DEFINITION
FHS	Filesystem Hierarchy Standard — the consistent directory layout Linux distributions follow.
SUID	A permission bit making an executable run with its file owner's privileges.
Sticky Bit	A permission bit restricting file deletion in a shared directory to the file's owner.
/etc/passwd	The world-readable file listing every user account on a system.
/etc/shadow	The root-only file storing hashed passwords.
sudo	A mechanism letting an authorized user run commands as another user, typically root, with logging.
journalctl	The command-line tool for querying the systemd journal.
ss	The modern command for viewing network sockets and connections (replaces netstat).
lsof	"List open files" — shows every file/socket/connection a process currently has open.
Persistence	A mechanism (cron, systemd service, SSH key, etc.) letting an attacker maintain access after initial compromise.
Baseline	A known-good reference state used to identify what's actually changed or anomalous.

End of guide. None of these commands are individually complex — the actual skill is knowing which one answers which question, and in what order, when you're standing in front of an unfamiliar host with limited time. That sequence, more than any single command, is what this guide was really trying to teach.